

1 Device Overview - Features

Texas Instrument MSP-EXP432E401Y

- ARM Cortex-M4 32-bit CPU with FPU (Floating point unit)
- Maximum CPU clock: 120MHz
- Design Bus Speed: 36MHz
- 256 Kb SRam, 1024 Kb Flash Memory
- I2C communication interface (Up to 3.33MHz)
- UART at 115200 bps for real-time serial communication
- Uses 1 onboard user button with debouncing and internal pull-up resistor
- Uses 3 user-controlled LEDs and multi-port digital I/O control pins

VL53L1X Time of Flight Sensor

- Operating voltage: 2.6V – 3.5V
- Ranges: short range of 136 cm, medium range of 290 cm and a long range of 360 cm. Up to 4m
- Max ranging frequency of 50Hz
- Up to 27° field of view (FoV)
- I2C Interface (up to 400kHz)

28BYJ-48 Stepper Motor

- Unipolar 4-phase stepper motor
- Operating voltage: 5V – 12V
- 512 total steps for a full 360° rotation
- Uses 11.25° (32 steps per revolution)
- Configured for full-step mode

ULN2003 Stepper Motor Driver

- Can handle 5-12V power input
- On-board 4 channel signal outputs

3D Visualization

- Receiver-side processing implemented using MATLAB for UART data acquisition, parsing, and 3D spatial visualization.

Cost Information

- MSP-EXP432E401Y Cost: ~\$69.90 CAD
- VL53L1X Time of Flight Sensor Cost: ~\$20CAD
- 28BYJ-48 Stepper Motor and ULN2003 Stepper Motor Driver: ~\$15
- Total Cost: ~\$104.9

1.2 Device Overview – General Description

This device provides a method for acquiring spatial distance measurements and generating a three-dimensional visualization of an indoor environment. The system integrates a Time-of-Flight distance sensor, stepper motor, serial communication and an external visualization software for mapping data. Distance measurements are taken through the VL53L1X Time-of-Flight (ToF) sensor, from the push of an on-board push button on the microcontroller, which measures the time required for emitted infrared light to reflect of an object and return to the sensor. This measured time is converted into distance using the formula:

$$\text{Measured Distance} = \frac{\text{Photon Travel time} * \text{Speed of Light}}{2}$$

The ToF sensor communicates with the microcontroller through an I2C communication interface, eliminating the requirement for analog signal conditioning. During operation, the sensor collects distances at 11.25° intervals as the stepper motor rotates the sensor through a full 360° rotation. A measurement status LED flashes at a distance reading. Following each measurement, the data is sent from the microcontroller to a PC using UART serial communication at a rate of 115.2kbps. On the PC, MATLAB helps store incoming serial data for visualization processing. Through trigonometry, it is converted from polar to cartesian form. The X coordinates are manually incremented for a depth simulation, and the Y and Z are used for the calculations. This transformation allows a three-dimensional plot representing the scanned environment.

1.3 Device Overview – Block Diagram

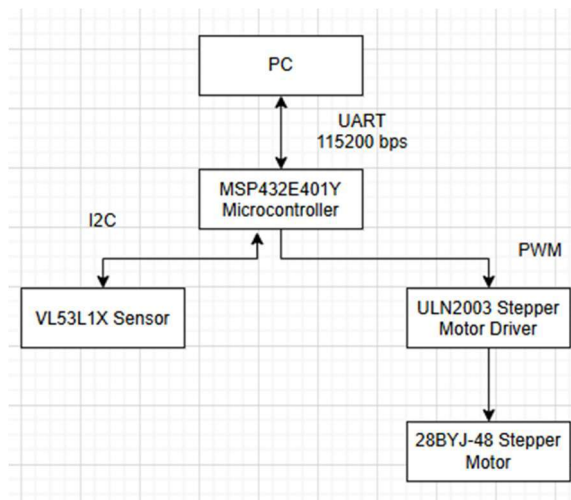


Figure 1. Block Diagram of Devices Used

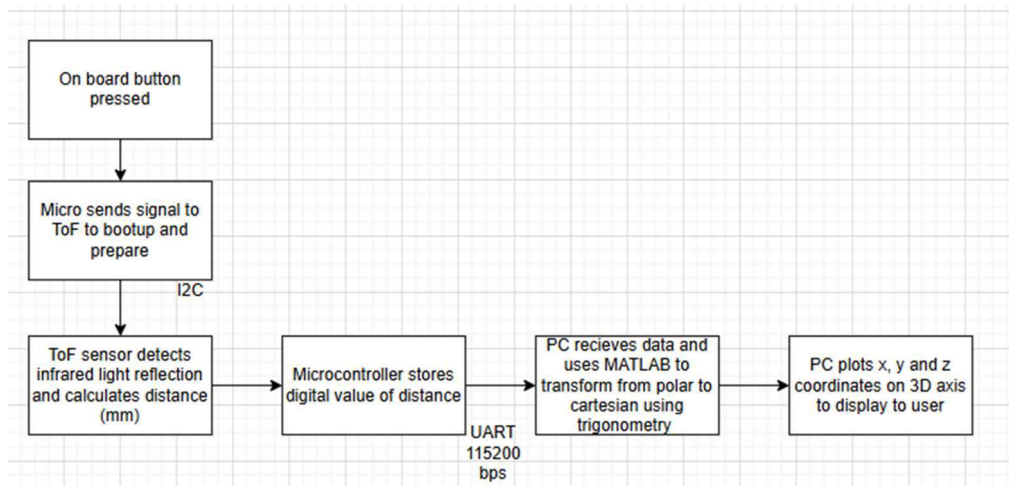


Figure 2. Block Diagram of Process

2 Device Characteristics

Parameter	Specification
MSP432E401Y Microcontroller	
System Bus Speed	36MHz
Measurement Status LED	Pin: PF0 (D4)
UART Transmission Status LED	Pin: PF4 (D3)
Additional Status LED (Scan Start)	Pin: PN1 (D1)
Start Button	Pin: PJ0
VL53L1X Time-of-Flight Sensor	
Vin	2.6V – 3.5V Operating Range: 3.3V Used
GND	Pin: GND
SDA	Pin: PB3
SCL	Pin: PB2
ULN2003 Driver Board	
In1	Pin: PH0
In2	Pin: PH1
In3	Pin: PH2
In4	Pin: PH3
V+	5V – 12V Operating Range: 5V Used
V-	GND
General System	
UART Communication Baud Rate	115200 bps
Angle Intervals	11.25°
Maximum Measurement Range	Up to 4m
Receiver Interface	PC with MATLAB
Standard I2C Transmission Rate	100kbps
Number of Data in a Scan	32

3 Detailed Description – Distance Measurement

Distance measurements were first obtained from the VL53L1X Time-of-Flight (ToF) sensor by mounting it on a rotating stepper motor. The sensor works by emitting a beam of infrared light using its Emitter. This beam of light reflects off an object, whose distance is being measured, and returns to the Receiver side of the sensor, all while the sensor measures the time. This measured time is used to determine the distance D according to the equation: $D = \frac{\text{flight time} \times \text{speed of light}}{2}$ where the speed of light is approximately 300,000 km/s. The sensor performs internal transduction, signal conditional, and analog-to-digital conversion, thus, providing a digital representation of the analog distance measurement to the microcontroller through an I2C serial communication interface.

During operation, the Keil code is flashed on the microcontroller. This code helps initialize GPIO pins, LEDs, protocols for UART and I2C, configuring the clock, the motor and ToF sensor. The ToF sensor is set to continuous ranging and long mode. When the onboard push button, PJ0, is pressed, the stepper motor can begin its process toward a full 360-degree scan. When scanning begins, the stepper motor, 28BYJ-48, rotates the sensor in the YZ-plane. This motor also requires 512 steps to complete a full 360-degree rotations. Therefore, a single angle/step of this is:

$$\frac{360^\circ}{512} = 0.703^\circ / \text{step}$$

Since an 11.25-degree interval is being used to take each measurement, the number of steps needed for that interval:

$$\frac{11.25^\circ}{0.703^\circ / \text{step}} = 16 \text{ steps}$$

A full rotation also produces:

$$\frac{360^\circ}{11.25^\circ} = 32 \text{ measurements}$$

Therefore, 32 samples per revolution are used for a full rotation. During each measurement cycle, the system checks if the ToF sensor has enough data needed, and if true, the data is retrieved and transmitted to the computer through UART communication. A status LED flashes to indicate a measurement has been successfully taken, as well as when UART is being used to transmit. Each distance measurement is associated with a corresponding angle value which is calculated by the number of motor steps completed and incremented by 11.25 degrees. For example, the first distance data in the cycle would be 0 degrees, the second would be 11.25 degrees, the third would be 22.5 degrees and so on until it returns to angle 0. This produces a circular set of measurements representing the scanned environment. After a full rotation is completed, the motor returns counterclockwise to its home position to ensure no wire tangling and prepares for the next scan if.

After the distance and angle values are transmitted to the PC through UART serial communication, the measurements are converted from polar to cartesian coordinates. This conversion is performed through MATLAB, where the following equations are used:

$$x = scan_index * slice_spacing$$

$$y = distance * \cos(\theta)$$

$$z = distance * \sin(\theta)$$

Where distance is the measured distance in millimeters, theta is the angle in rad, scan_index is the scan number, and slice_spacing is the chosen displacement between scans for depth. For instance, if the second scan is going on, the displacement is going to be multiplied by 2 to indicate the displacement has been increased by 2 times the amount to add depth and mimic movement. The angle value is converted from degrees to radians before applying the trigonometric functions.

For example, given distance = 1000 mm, angle = 30 degrees, scan number is 1, and the displacement is chosen as 100 mm:

$$\theta = 30^\circ * \frac{\pi}{180} = 0.524 \text{ rad}$$

$$y = 1000 \cos(0.524) = 866 \text{ mm}$$

$$z = 1000 \sin(0.524) = 500 \text{ mm}$$

$$x = scan_index * slice_spacing = 1 * 100 = 100 \text{ mm}$$

Therefore, the final coordinate would be (x,y,z) = (100, 866, 500).

To create a three-dimensional scan, multiple 360-degree scans are performed at different positions along the x-axis. After each full scan of 360-degrees, the device manually moves forward by a fixed distance similar to the chosen displacement. MATLAB uses the variable slice_spacing_mm to indicate that chosen displacement. Each scan is assigned an index beginning from zero, so that Scan 0's x coordinate would be 0 mm, Scan 1's x coordinate would be slice_spacing_mm, Scan 2's x coordinate would be 2*slice_spacing_mm, and so on. This allows the circular scans to be stacked along the x-axis, forming a three-dimensional environment.

3.2 Detailed Description – Visualization

The visualization of the environment is performed using MATLAB. MATLAB receives the distance measurements transmitted from the microcontroller, converts them into coordinates for the three dimension and plots them on a 3-D axis for a model of the scanned location.

MATLAB begins by configuring serial communication from the PC and microcontroller. This is done using the `serialport()` function which establishes communication via a COM port and baud rate specified in the code. The communication is defaulted to 115200 to match the UART configuration of the microcontroller.

The `configureTerminator()` function in the code defines characters for the end of line allowing a correct interpretation of data lines. The `flush()` function clears any previous data stored in the serial buffer before new data is received (see Figure 3).

```
configureTerminator(device, "CR/LF");  
flush(device);
```

Figure 3. MATLAB configureTerminator() and flush() Functions

After the user presses button PJ0 on the microcontroller, the scan begins and MATLAB reads incoming serial data using the `readline()` function. Each data contains three values: scan index, angle (degrees), distance (millimeters). This received data is processed via the `sscanf()` function which takes the numerical values from each line of text and then stores them in a matrix called *results*. Each row within the matrix represents one measurement.

After all required measurements are received, the data is converted into three-dimensional Cartesian coordinates (for detailed equations, see page 6). The angle values of the scans are first converted from degrees to radians using the `deg2rad()` function in MATLAB. The cartesian coordinates are then calculated using trigonometric relationships based on the distance and angle. The x-coordinate represents the manual displacement between scans, adding depth, whereas the y and z coordinates represent the circular scan plane. These values are stored into a matrix called *points*.

The system then generates a 3-D plot using MATLAB's visualization features. The `scatter3()` function plots individual coordinate points in the 3-D plane. Furthermore, the `grid()` function is used to display grid lines. The adjacent points are then connected using the `plot3()` function. Additional `plot3()` commands are used to connect points between neighboring scans (see Figure 4). Finally, the axis labels and titles are added via `xlabel()`, `ylabel()`, `zlabel()` and `title()` functions. This produces vertical connected between the scans creating a spatial representation of the scanned environment.

4 Applications Example

A rotational time-of-flight scanner uses LiDAR (Light Detection and Ranging) to collect spatial information and generate three-dimensional spatial representations of environments. One of the primary uses of LiDAR is mapping, where it can measure spaces such as corridors, rooms or complex structures. These measurements can be combined into layered scan slices to create a three-dimensional representation. This technology is important for planning interior layouts, verifying structural dimension, uncovering hidden structural features or analyzing spaces.

This system is also useful for robot navigation and environmental awareness. Autonomous systems often rely on distance sensing technologies to detect obstacles and ensure safety. A rotating ToF sensor allows measurements to be collected across a wide field of view, enabling detection of surfaces to avoid collisions. The ability to generate three-dimensional visualizations allows the measured data to be interpreted visually, improving the understanding of surrounding physical structures.

4.2 Instructions

1. Configure the circuit by following the circuit schematic shown in Figure 7 (page 14)
2. Ensure that the microcontroller is plugged into a PC and determine the COM port of the PC by looking through the 'Device Manager' software. Once in the software, scroll down to 'Ports' and expand the arrow. When expanded, note down the number # in the statement: 'XDS110 Class Application/User UART (COM #)'.
3. Open the MATLAB program used for serial data collection and confirm the correct COM port is selected by changing line 6 'port = "COM#";' if needed. Ensure that the baud rate in line 7 is 115200.
4. In line 9, change SLICE_SPACING_MM variable to whichever displacement is needed.
5. Open the Keil project code. Select 'Options for Target' then 'C/C++' and then select optimization to '-O0'. Furthermore, go to the 'Debug' tab, select 'Settings' and 'CMSIS-DAP Debugger', then 'Settings' and ensure the top left says 'XDS110 with CMSIS-DAP'. Then click 'OK' when done.
6. Change the number of scans in line 10 to however many needed. Default is 3.
7. Translate, Build and Load the code from Keil.
8. Press the RESET button on the microcontroller to flash the project.
9. Run the MATLAB script.
10. Wait for the program to display the message prompting the user: 'Press Enter here, then press PJ0 on the board to start scanning'. LED D1 will light up.
11. Press the onboard push button labelled PJ0. The stepper motor will begin rotating.
12. Make sure the stepper motor stays still when it collects a full 360-degree scan. Distances will be collected every 11.25 degrees with a measurement LED flashing each time a measurement is being took.

13. Move the device forward manually by the displacement amount specified in line 9 for SLICE_SPACING_MM when the device is moving counterclockwise to home after a full 360-degree scan. Each time UART transmits data, LED D3 will light up.
14. Repeat steps 12-13 for each full scan.
15. When the scans are completed, the system will stop transmitting data. LED D1 will turn off. MATLAB will finish, display the received measurements and plot. Ensure it aligns with the environment scanned.

4.3 Expected Output

The scanning location was J, in the John Hodgins Engineering Building. Below is a picture of the hallway in real life and a comparison with the produced scan. The x-axis (depth) represented the forward displacement between consecutive scans. Each scan is assigned a new x-position based on the distance specified and scan number. The y-axis represents the horizontal direction of the scan. It corresponds to the left-right direction relative to the sensor. Lastly, the z-axis represents the vertical direction of the scan. It corresponds to the height of measured surfaces relative to the sensor.

As can be seen from Figure 5 and Figure 6, there are irregular points visible. These points are caused due to reflective surfaces such as windows (see Figure 4) that cause variability in the detected distance values. However, the overall shape of hallway remains visible.

Scanned Location:

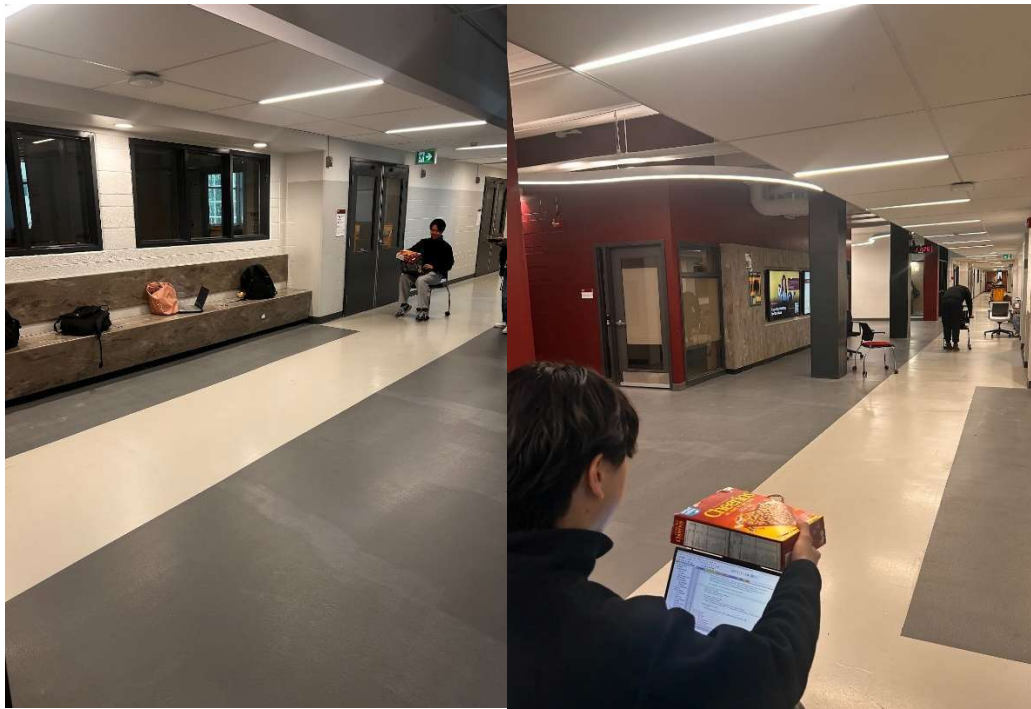


Figure 4. End to Front View on the Left and Front to End View on the Right Hallway Scan

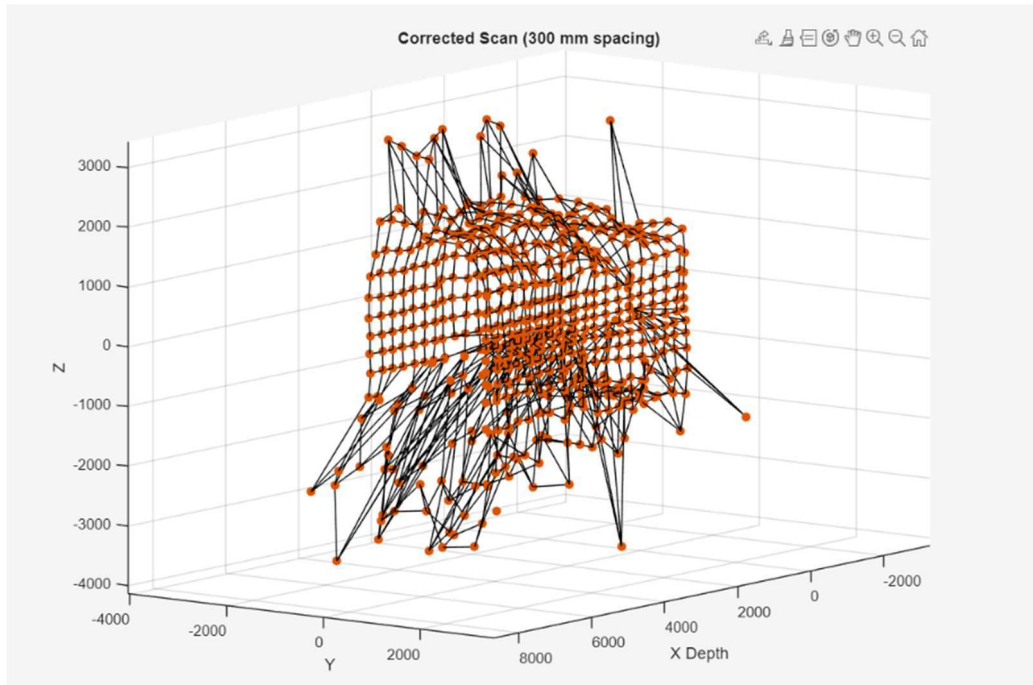


Figure 5. Right Side Birds Eye View of Hallway

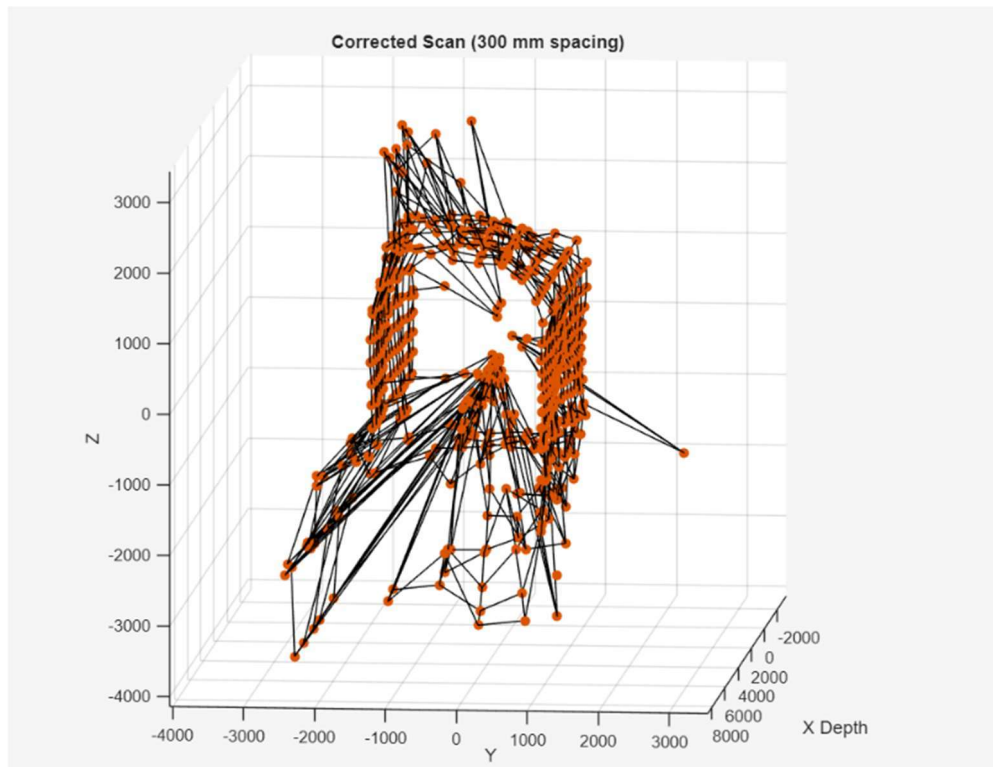


Figure 6. Front View of Hallway

5 Limitations

The microcontroller has a limited floating point processing capability compared to computing systems. It features a 32-bit Floating Point Unit. This means that the microcontroller can fully support single precision arithmetic. The FPU can also convert between fixed- and floating-point data. Even though this means that the microcontroller can handle trigonometric operations for 32-bit precision, there is still a risk at losing accuracy. Floating point rounding limitations can introduce minor inaccuracies in coordinate calculations and when combined with repeated trigonometric calculations, these rounding effects can produce small deviations in calculated coordinate points. Therefore, the trigonometric functions were accomplished through the PC and MATLAB ensuring 64-bit precision and faster speeds, reducing computational load on the microcontroller.

The maximum standard serial communication rate supported for the used PC is 128kbps. This was verified by going to Windows 11 Device Manager, checking 'Port Settings' of the 'XDS110 Class Application/User UART' device where 128kbps was the largest option. For this system however, the UART communication was at a rate of 115200 bps.

The communication method used between the microcontroller and PC is UART, and between the microcontroller and the ToF sensor is I2C (Inter-Integrated Circuit). The microcontroller is the leader, and the ToF sensor is the follower. A speed of up to 100kbps is used for I2C communication, which is the standard mode. However, the maximum fast mode for the I2C is up to 400kHz, and high-speed mode is up to 3.33Mbps. The actual clock frequency is around 30kHz.

The primary limitations on speed are the stepper motor rotation time delay and the ToF sensor initialization and begin ranging. The motor delay in the Keil code is defined as 200000 cycles, and with a bus speed of 36MHz: $t_{delay} = \frac{200000}{36M} = 5.56 \text{ ms}$. For each step, there are four full step states so one step requires $4 * 5.56 = 22.22 \text{ ms}$. With 16 motor steps for a shaft rotation of 11.25 degrees, it would be $16 * 22.22 = 355.5 \text{ ms}$. This means that it requires the motor 0.356s to move between angles. Per rotation this is: $32 \text{ measurements} * 355.5 \text{ ms} = 11.38 \text{ s}$. Therefore, the stepper motor is the biggest limitation on speed, causing 11.38s of delay within a rotation.

The maximum quantization error is the same as the resolution (1 LSB) of the system. This is defined as:

$$\Delta = \frac{\text{Maximum output}}{2^n}$$

Given, the maximum ranging distance output is 4m and n is 16 bits that the ToF sensor sends to the microcontroller:

$$\Delta = \frac{4000 \text{ mm}}{2^{16}} = 0.06\text{mm}$$

Therefore, 0.06mm is the resolution and maximum quantization error.

The assigned bus speed is 36MHz, which was configured through modifying PLL.c and PLL.h files. The system clock frequency was calculated using:

$$SysClk = \frac{f_{VCO}}{PSYSDIV + 1}$$

PSYSDIV is being used as 9 for a clean division, therefore a f_{VCO} needs to be 360MHz for a SysClk of 36MHz. Where:

$$f_{VCO} = \frac{f_{XTAL}}{(Q + 1)(N + 1)} * MINT$$

f_{XTAL} is given 25MHz, Q is 0, N is 4 and MINT is chosen 72.

$$f_{VCO} = \frac{25 \text{ MHz}}{(0 + 1)(4 + 1)} * 72 = 360\text{MHz}$$

Therefore:

$$SysClk = \frac{360\text{MHz}}{9 + 1} = 36\text{MHz}$$

The bus speed is therefore able to be 36MHz.

In the project specification file, Table 2 represents the status LEDs. In the context of this report, since 6 is the second least significant digit of the student number, the measurement status LED is PF0 which flashes when a valid distance measurement is received from the ToF sensor. The UART transmission status LED is PF4, which indicates when data is being transmitted to the PC through UART communication. Lastly, the additional status indicator PN1 indicates an active scanning operation and turns off when scans are completed.

6 Circuit Schematic

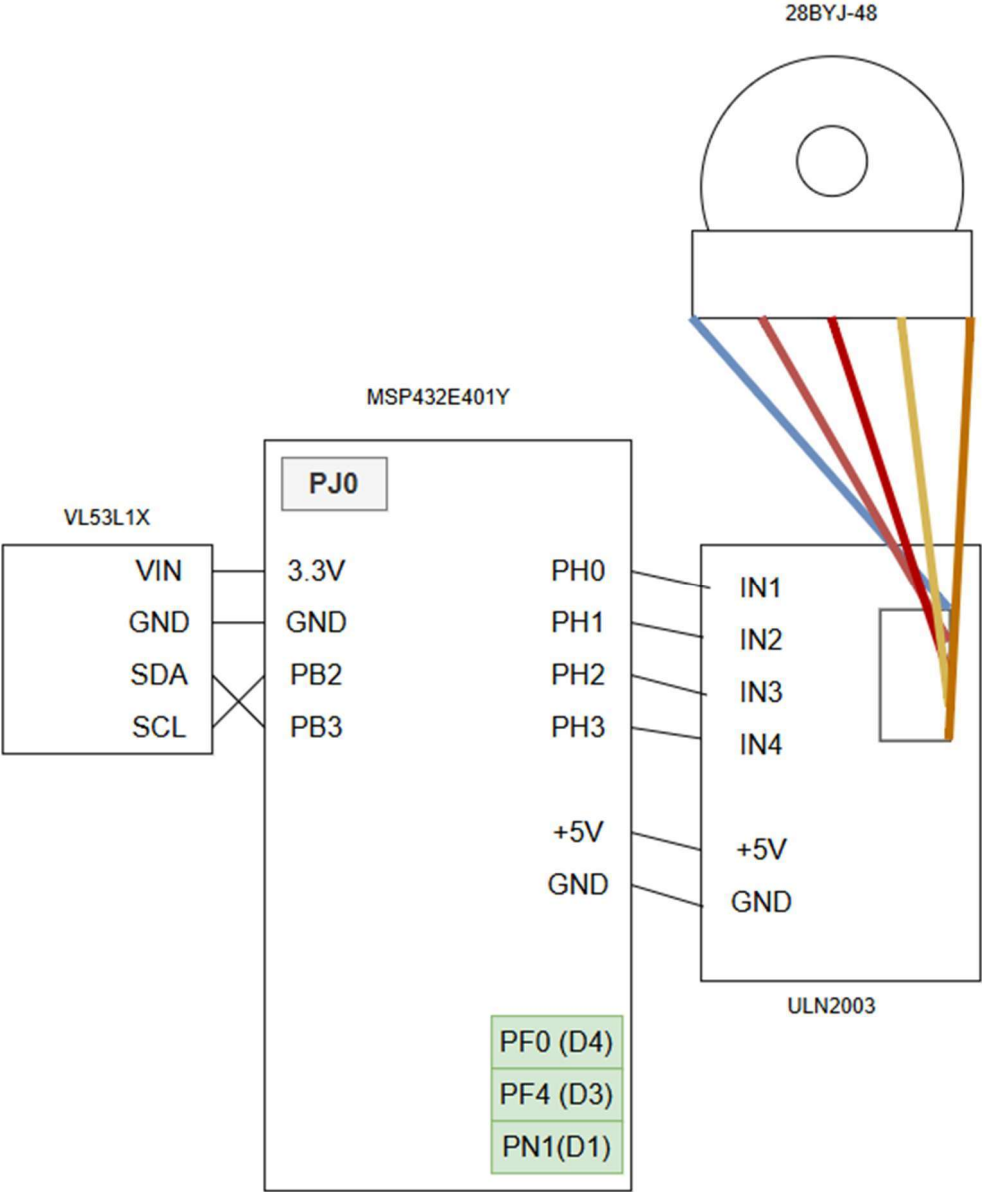


Figure 7. Circuit Schematic of the Entire System

7 Programming Logic Flowchart(s)

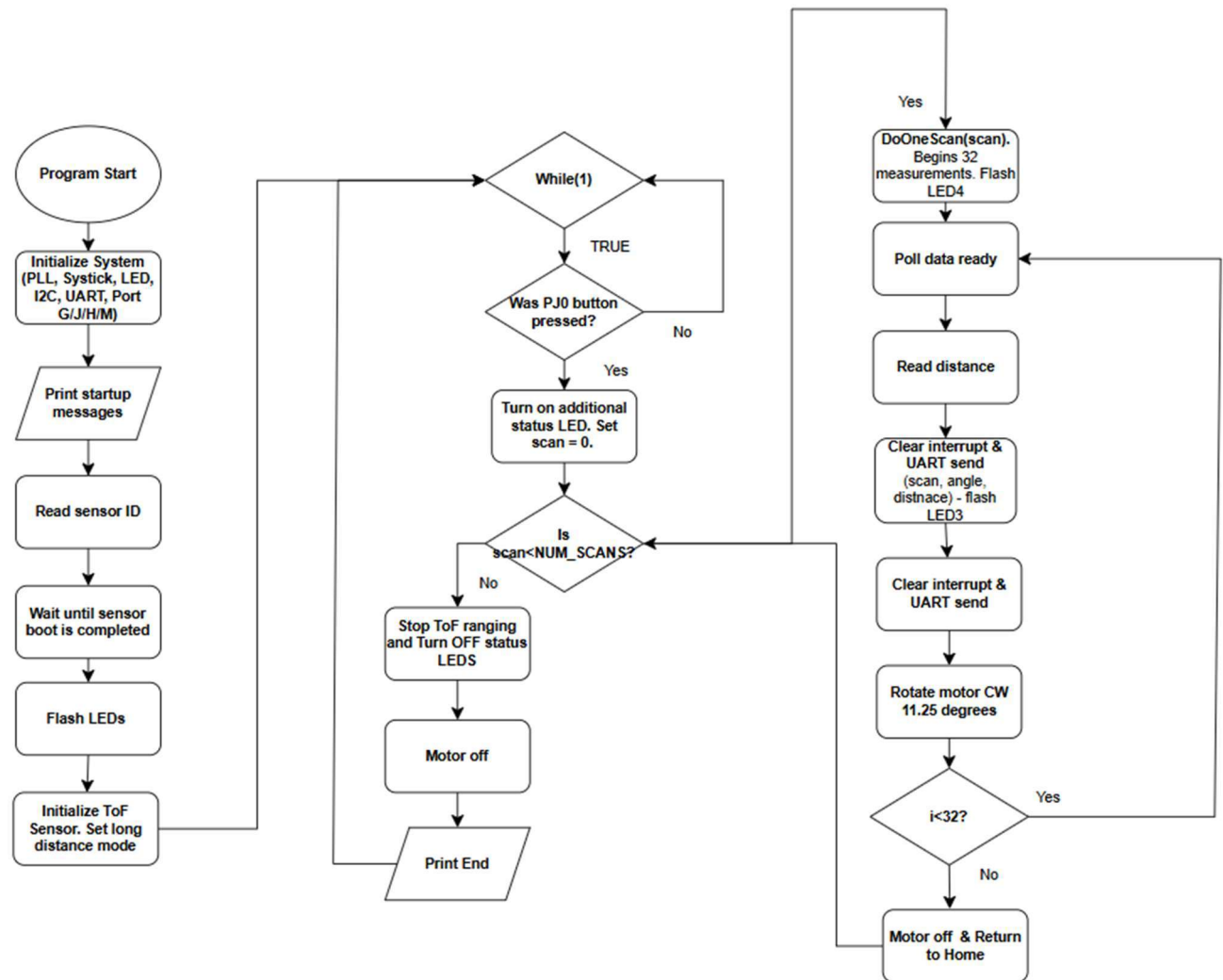


Figure 8. Keil Code (.c File) Flowchart

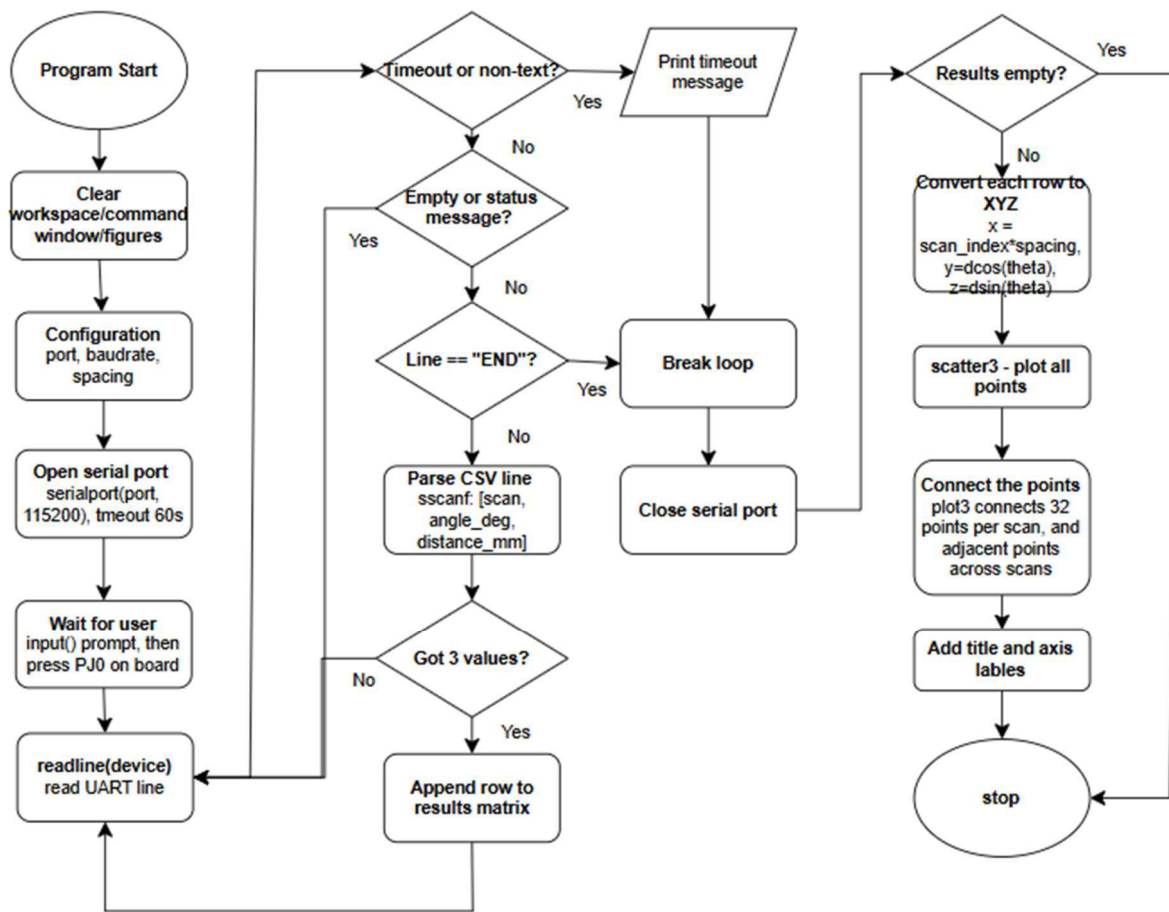


Figure 9. MATLAB (.m File) Code Flowchart